

C01 — R, RStudio i własna przestrzeń pracy

Zarządzanie informacją — część ilościowa — 2026/27

Marek Deja

2026/27

Spis treści

Cel i rytm zajęć	1
Przygotowanie konta przed zajęciami	1
Początek zajęć w RStudio	2
Konsola, skrypt i komunikaty	2
R jako kalkulator	2
Typy danych	3
Obiekty: przechowywanie wartości	3
Wektory: kilka wartości	4
Warunki i braki	4
Tabela: wiersze i kolumny	4
Pomoc i odtwarzanie	5
Zadanie Z01 — czas znalezienia informacji	5
Kontrola, oddanie i zakończenie	6
Instalacja na własnym komputerze — poza rdzeniem zajęć	6
Pytania do samokontroli	6

Cel i rytm zajęć

Po zajęciach rozróżnisz polecenie, obiekt i wynik, zapiszesz kod w skrypcie oraz odczytasz element wektora i kolumnę tabeli. Pobierzesz własny projekt, a po pracy otrzymasz potwierdzenie wysyłki. Nie trzeba dziś znać testów statystycznych ani poleceń Gita.

Czas	Praca
0–10 min	Logowanie, pobranie i otwarcie projektu
10–30 min	Konsola, skrypt, obiekty i typy
30–55 min	Wektory i mała tabela — wspólnie
55–80 min	Jedno zadanie samodzielne Z01
80–90 min	Kontrola, wysyłka, status i wylogowanie

Przygotowanie konta przed zajęciami

Otwórz rejestrację GitHub. Wybierz login, podaj e-mail i ustaw hasło na stronie. Potwierdź e-mail. Konto osobiste jest bezpłatne; zapamiętaj login, bo nie jest tym samym co nazwa projektu RStudio.

W ustawieniach GitHub wybierz **Password and authentication** i skonfiguruj uwierzytelnianie dwuskładnikowe. Sprawdź logowanie i przechowuj kody odzyskiwania na własnym urządzeniu. Hasła, tokeny i kody odzyskiwania nie należą do raportu ani repozytorium.

Prześlaz login prywatnym kanałem kursu. Otrzymasz pseudonim, np. `s017`, oraz zaproszenie do własnego prywatnego repozytorium. Przyjmij je i sprawdź w przeglądarce, czy widzisz repo. Przykładowe ID w instrukcji zastąp własnym; nie używaj numeru albumu.

Początek zajęć w RStudio

R wykonuje obliczenia; RStudio jest środowiskiem, w którym edytujesz skrypt, oglądasz pliki i odczytujesz wyniki. W sali R i pakiet są już zainstalowane. Zapisz pliki na bieżąco: stan pracy i Gita jest czyszczony między spotkaniami, więc trwałą kopią jest potwierdzona wersja w GitHub.

Otwórz RStudio. W dolnym panelu **Console** wpisz poniższe polecenia, zastępując ID i adres. Logowanie otworzy GitHub lub pokaże adres i jednorazowy kod; hasło oraz drugi składnik wpisujesz na GitHub. Po potwierdzeniu R pokaże login konta.

```
library(badaniaZI)
sprawdz_srodowisko()
zaloguj_github()
rozpocznij_zajecia("C01", id_studenta = "s017",
  repo_url = "https://github.com/prowadczy/badania-s017.git")
```

Pobieranie tworzy nowy katalog `moje-badania` i otwiera projekt `.Rproj`. Nazwa autora commitów jest ustawiana lokalnie dla tego projektu, a nie dla wszystkich użytkowników komputera. Na początku kolejnych zajęć wykonasz te same kroki i odzyskasz ostatnią wysłaną pracę. Jeśli używasz Positrona, otwórz pobrany katalog i główny projekt, a polecenia wykonuj w jego konsoli R.

Gdy repo nie jest jeszcze dostępne, możesz ćwiczyć lokalnie. Funkcja poniżej tworzy nowy projekt i syntetyczne dane; nie nadpisze istniejącego katalogu. Otwórz plik `.Rproj` z utworzonego katalogu. Lokalne przygotowanie nie jest potwierdzonym oddaniem.

```
utworz_projekt("s017", "S02", katalog = "moje-badania-lokalnie")
```

Konsola, skrypt i komunikaty

W konsoli znak `>` oznacza gotowość na polecenie; nie przepisuj go do kodu. Znak `+` na początku nowej linii oznacza, że R czeka na koniec polecenia, np. brakujący nawias albo cudzysłów. Klawisz Esc przerywa niedokończone polecenie, aby można je było poprawić.

Otwórz **File** → **New File** → **R Script** lub zapisany `zadania/z01/analiza.R`. Uruchamiaj linię skrótem `Ctrl+Enter` (na macOS `Cmd+Enter`), a zapisuj `Ctrl+S/Cmd+S`. Tekst po `#` jest komentarzem; nie wykonuje obliczenia. Panel **Environment** pokazuje bieżące obiekty, **Files** pliki, a **Plots** wykresy.

Komunikat `object 'czasy' not found` zwykle oznacza, że nie wykonano linii tworzącej obiekt albo nazwa została napisana inaczej. Zapis skryptu nie wykonuje kodu automatycznie. Pod koniec pracy uruchom go od początku, aby sprawdzić kolejność poleceń.

R jako kalkulator

Zaczynamy od najprostszej roli R: kalkulatora. Dzięki temu widać, że każde polecenie ma składnię, a wynik pojawia się od razu pod kodem. Patrzymy na to, czy R wykonał działanie bez błędu i czy wynik ma sens matematyczny.

```
# Dodawanie dwóch liczb.
5 + 3
```

```
## [1] 8
```

```
# Potęgowanie: 2 do czwartej potęgi.
```

```
2^4
```

```
## [1] 16
```

```
# Pierwiastek kwadratowy ze 144.
```

```
sqrt(144)
```

```
## [1] 12
```

```
# Logarytm naturalny z 10.
```

```
log(10)
```

```
## [1] 2.302585
```

Typy danych

R musi wiedzieć, czy pracuje z liczbą, tekstem czy kategorią. Ten sam zapis może wyglądać podobnie dla człowieka, ale dla programu oznaczać inny typ danych. Patrzymy na wyniki `class()` i `typeof()`, bo one mówią, jakie operacje będą możliwe na danym obiekcie.

```
# Liczba: można ją dodawać, odejmować i uśredniać.
```

```
czas <- 12
```

```
class(czas)
```

```
## [1] "numeric"
```

```
typeof(czas)
```

```
## [1] "double"
```

```
# Tekst: opisuje kategorię, ale nie nadaje się do obliczania średniej.
```

```
kanal <- "WWW"
```

```
class(kanal)
```

```
## [1] "character"
```

```
typeof(kanal)
```

```
## [1] "character"
```

```
# Wartość logiczna: TRUE albo FALSE.
```

```
czy_www <- kanal == "WWW"
```

```
class(czy_www)
```

```
## [1] "logical"
```

```
czy_www
```

```
## [1] TRUE
```

Obiekty: przechowywanie wartości

W instytucji wracamy do tej samej wartości, np. liczby dokumentów w kolekcji. Obiekt pozwala zapisać ją pod nazwą i wykorzystać później. Operator `<-` przypisuje wartość po prawej stronie do nazwy po lewej.

```
liczba_dokumentow <- 45
```

```
liczba_dokumentow
```

```
## [1] 45
```

```
po_dodaniu <- liczba_dokumentow + 5
po_dodaniu
```

```
## [1] 50
```

Nazwy: czasy, Czasy i CZASY są różnymi obiektami. Używaj małych liter i podkreśleń, bez spacji. Samo przypisanie nie musi drukować wyniku; wpisanie nazwy go pokaże.

Wektory: kilka wartości

Wektor przechowuje serię wartości tego samego typu, np. czasy szukania informacji. Funkcja `c()` łączy wartości, a nawiasy kwadratowe wskazują ich pozycje. Numerowanie zaczyna się od 1; dodanie liczby do wektora działa na każdy element.

```
czasy_szukania <- c(15, 22, 45, 12, 60)
czasy_szukania[3]
```

```
## [1] 45
```

```
czasy_szukania + 5
```

```
## [1] 20 27 50 17 65
```

Warunki i braki

Warunek zwraca `TRUE` albo `FALSE`, np. czy czas przekracza 30 minut. Podwójny znak `==` porównuje wartości; `<-` tworzy lub zmienia obiekt. `NA` oznacza brak znanej wartości i nie jest zerem.

```
czasy_szukania > 30
```

```
## [1] FALSE FALSE TRUE FALSE TRUE
```

```
(czasy_szukania > 20) & (czasy_szukania < 50)
```

```
## [1] FALSE TRUE TRUE FALSE FALSE
```

```
z_brakiem <- c(15, NA, 45)
is.na(z_brakiem)
```

```
## [1] FALSE TRUE FALSE
```

```
mean(z_brakiem)
```

```
## [1] NA
```

```
mean(z_brakiem, na.rm = TRUE)
```

```
## [1] 30
```

Ostatnie dwa wyniki różnią się, ponieważ pierwszy zachowuje brak, a drugi pomija go w obliczeniu. Nie usuwamy tu całej osoby z danych: pomijamy tylko nieznaną wartość w określonej operacji. Reguły braków przećwiczymy dokładniej w C02.

Tabela: wiersze i kolumny

Ramka danych przypomina arkusz: wiersz jest obserwacją, kolumna zmienną. Tworzymy małą tabelę ręcznie, aby zobaczyć strukturę. Znak `$` odczytuje nazwaną kolumnę, a `tabela[wiersz, kolumna]` wskazuje pozycję w dwóch wymiarach.

```
uzytkownicy <- data.frame(
  id = c("o001", "o002", "o003"),
  kanal = c("WWW", "e-mail", "WWW"),
  czas = c(12, 8, 15)
```

```

)
uzytkownicy

##      id  kanal  czas
## 1 o001    WWW   12
## 2 o002 e-mail    8
## 3 o003    WWW   15

str(uzytkownicy)

## 'data.frame':   3 obs. of  3 variables:
## $ id   : chr  "o001" "o002" "o003"
## $ kanal: chr  "WWW"  "e-mail" "WWW"
## $ czas : num  12 8 15

nrow(uzytkownicy)

## [1] 3

uzytkownicy$czas

## [1] 12 8 15

uzytkownicy[2, "kanal"]

## [1] "e-mail"

```

Tabela ma trzy wiersze. Kolumna **czas** jest wektorem liczbowym, a **kanal** wektorem tekstowym. Nie obliczamy średniej z nazw kanałów. Później skala pomiarowa pomoże dobrać opis i test; klasa R jest informacją techniczną, która nie zastępuje skali.

Pomoc i odtwarzanie

Nie trzeba pamiętać wszystkich argumentów. Wpisz `?mean` w konsoli i sprawdź argument `na.rm`; nie wpisuj znaku zapytania do raportu jako wyniku analizy. Nazwy plików i katalogów sprawdzaj w panelu Files. Otwarcie głównego `.Rproj` ustala katalog projektu, więc nie potrzebujesz `setwd()`.

Plik `.R` zawiera kod; `.md` własne odpowiedzi i odnośniki do wyników; `.Rmd` łączy tekst z wykonywalnym kodem i może tworzyć HTML oraz PDF. Gotowe materiały obu formatów są w pakiecie i Pages. Do zwykłego zadania nie potrzebujesz LaTeX ani renderowania PDF.

Zadanie Z01 — czas znalezienia informacji

W pliku `zadania/z01/analiza.R` utwórz wektor **czasy** z wartościami 6, 9, 4, 12, 8 minut. Odczytaj drugi element i oblicz średnią. Utwórz trzywierszową ramkę **proby** z kolumnami **id**, **kanal** i **czas**, używając dwóch kanałów WWW/e-mail; odczytaj jej kolumnę **czas** i liczbę wierszy. Usuń instrukcję `stop()` dopiero po uzupełnieniu kodu.

W `odpowiedzi.md` wpisz otrzymane liczby i wyjaśnij w 250–450 słowach różnicę między konsolą a skryptem oraz źródło jednego wyniku. Opisz, jak sprawdzisz odbiór własnej pracy i zdalne SHA. Nie wpisuj bieżącego SHA do pliku, który sam wchodzi do tego commitu: odczytaj je po wysyłce z pokwitowania.

Kryterium	Punkty
Z01.1: zapisany projekt i działający skrypt z obiektami	3
Z01.2: poprawny odczyt wektora/tabeli i obliczenie	3
Z01.3: objaśnienie konsoli, skryptu i źródła wyniku	2
Z01.4: własne repo/ID oraz wskazanie pokwitowania SHA/statusu	2

Pełne poziomy punktacji znajdziesz przez `rubryka("Z01")` i w katalogu rubryk. Kontrola techniczna nie zastępuje oceny uzasadnienia.

Kontrola, oddanie i zakończenie

Zapisz oba pliki i uruchom skrypt od początku. Kontrola pokaże tabelę: wiersz `FALSE` wskazuje element do poprawienia. Jeżeli kontrola przejdzie, oddanie wykona commit i push oraz zapisze na GitHub pokwitowanie z SHA i czasem serwera; lokalny commit nie wystarczy.

```
sprawdz_zadanie("Z01", katalog = ".")
oddaj_zadanie("Z01", katalog = ".")
status_oddania("Z01", katalog = ".")
pobierz_ocene("Z01", katalog = ".")
wyloguj_github()
```

Brak opublikowanej oceny nie oznacza zera. Przy braku sieci zachowaj kopię własnych plików na prywatnym nośniku przed czyszczeniem komputera i ponów wysyłkę po odzyskaniu połączenia. Po potwierdzeniu oddania zakończ logowanie pakietu, wyloguj przeglądarkę w sali i zamknij RStudio bez zapisywania `.RData`.

Instalacja na własnym komputerze — poza rdzeniem zajęć

W sali nie instalujesz pakietu na początku każdego spotkania. Na własnym komputerze zainstaluj R i RStudio, a następnie `remotes` z CRAN oraz pakiet kursu z GitHub. Instalowanie i logowanie są odrębnymi czynnościami; publiczny pakiet można pobrać bez dostępu do prywatnego repo pracy.

```
install.packages("remotes")
remotes::install_github("MarekDejaUJ/projektowanie-badan-ZI",
  dependencies = NA, upgrade = "never")
```

Wersję rocznika sprawdzisz przez `packageVersion("badaniaZI")`. GitHub CLI instaluje informatyk w sali; na własnym komputerze jest potrzebny dla wariantu logowania `gh`. Alternatywny device flow korzysta z aplikacji kursu, jeśli jest skonfigurowana.

Pytania do samokontroli

Dlaczego zapisany skrypt może zwrócić `object not found`? Czym różnią się `NA` i zero? Dlaczego przypisanie nie jest porównaniem? Co potwierdza, że praca dotarła do GitHub? Wskaż odpowiedzi na własnym projekcie.